LAPORAN PRAKTIKUM

MATEMATIKA DISKRIT LANJUT

MODUL 2: PROPERTI, REPRESENTASI, DAN ISOMORFISME GRAF

Laporan Ini Disusun Untuk Memenuhi Tugas Praktikum
Mata Kuliah : Matematika Diskrit Lanjut



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

BAB I PENDAHULUAN

Modul 2 membahas graf khusus, properti dasar graf, cara merepresentasikan graf, dan isomorfisme. Tujuannya agar bisa membuat graf lengkap, graf lingkaran, graf roda, graf bipartit, dan graf berarah, menghitung derajat dan memverifikasi Handshaking Theorem, membuat adjacency list, adjacency matrix, dan incidence matrix, serta memeriksa isomorfisme pada studi kasus.

Task01 dan Task02 dikerjakan individu. Task03 sampai Task05 adalah studi kasus kelompok yang di kelas dibagi satu soal per anggota, dan pada laporan ini ketiganya ikut dikerjakan. Program dijalankan dengan Python 3.14.6, NetworkX 3.6.1, dan Matplotlib 3.10.8.

Tabel 1.1: Isi arsip pengumpulan

Berkas	**Isi**
`[MDL2026_2B_D4_2025]_Modul2_048/`	`Task01_Individu.py` sampai `Task05_Datasister.py`
`[MDL2026_2B_D4_2025]_Modul2_048.pdf`	Laporan praktikum ini
`[MDL2026_2B_D4_2025]_Modul2_048.docx`	Laporan praktikum ini dalam format Word
`[MDL2026_2B_D4_2025]_Modul2_048.txt`	Dokumentasi referensi resmi dan tanya jawab selama pengerjaan

BAB II TASK01: GRAF LENGKAP, LINGKARAN, DAN RODA

2.1 Kode Program

```
def properti(G, nama):
    print(f"--- {nama} ---")
    print(f"Jumlah simpul : {G.number_of_nodes()}")
    print(f"Jumlah sisi   : {G.number_of_edges()}")
    print("Derajat       : " + ", ".join(f"{v} = {d}" for v, d in G.degree()))
    print()


K4 = nx.complete_graph(4)
C5 = nx.cycle_graph(5)
# W6 menurut Rosen adalah C6 ditambah satu simpul pusat, jadi totalnya 7 simpul.
# wheel_graph(n) di NetworkX sudah menghitung simpul pusat, makanya dipakai wheel_graph(7).
W6 = nx.wheel_graph(7)

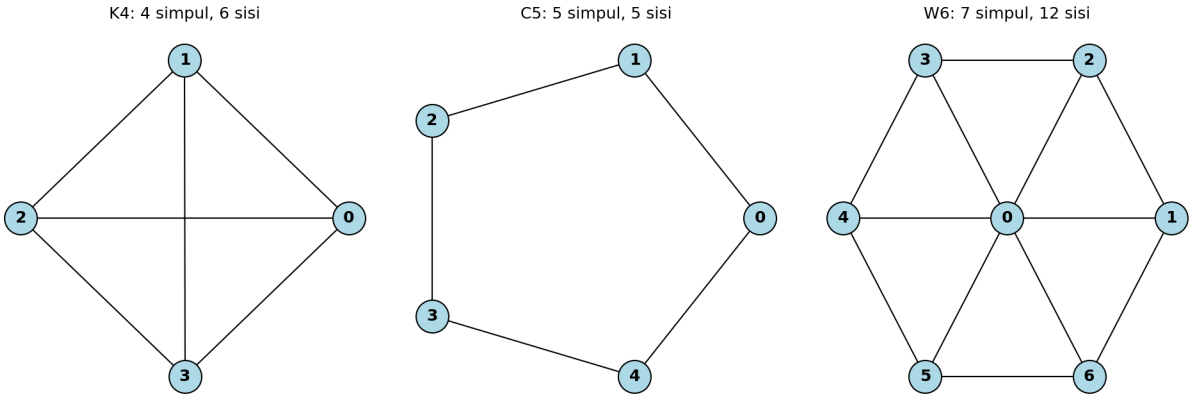
for G, nama in [(K4, "Graf lengkap K4"), (C5, "Graf lingkaran C5"), (W6, "Graf roda W6")]:
    properti(G, nama)
```

2.2 Output Program

```
--- Graf lengkap K4 ---
Jumlah simpul : 4
Jumlah sisi   : 6
Derajat       : 0 = 3, 1 = 3, 2 = 3, 3 = 3

--- Graf lingkaran C5 ---
Jumlah simpul : 5
Jumlah sisi   : 5
Derajat       : 0 = 2, 1 = 2, 2 = 2, 3 = 2, 4 = 2

--- Graf roda W6 ---
Jumlah simpul : 7
Jumlah sisi   : 12
Derajat       : 0 = 6, 1 = 3, 2 = 3, 3 = 3, 4 = 3, 5 = 3, 6 = 3
```



Gambar 2.1: Visualisasi K4, C5, dan W6

2.3 Lesson Learnt

K4 punya 6 sisi karena setiap pasang dari 4 simpul terhubung, sesuai rumus n(n-1)/2. C5 punya sisi sebanyak simpulnya dan semua simpul berderajat 2.

Yang perlu diwaspadai adalah `nx.wheel_graph(n)`. Parameter n di NetworkX adalah jumlah seluruh simpul termasuk pusat, sedangkan W_n menurut Rosen adalah C_n ditambah satu simpul pusat sehingga punya n + 1 simpul. Kalau langsung ditulis `nx.wheel_graph(6)`, yang terbentuk adalah W5 dengan 6 simpul dan 10 sisi. W6 yang benar dibuat dengan `nx.wheel_graph(7)`: 7 simpul dan 12 sisi, pusat berderajat 6 dan simpul lain berderajat 3.

BAB III TASK02: DERAJAT DAN REPRESENTASI GRAF

3.1 Kode Program

Soal tidak menyebut graf yang harus dianalisis, jadi yang dipakai tiga graf dari Task01.

```
def cetak_matriks(M, label):
    print("     " + " ".join(f"{v:>2}" for v in label))
    for v, baris in zip(label, M):
        print(f"  {v:>2} " + " ".join(f"{int(x):>2}" for x in baris))


def analisis(G, nama):
    print(f"===== {nama} =====")
    derajat = dict(G.degree())
    print("Derajat tiap simpul:", derajat)

    total = sum(derajat.values())
    m = G.number_of_edges()
    print(f"Jumlah derajat = {total}, 2 x |E| = 2 x {m} = {2 * m}")
    print("Handshaking Theorem terbukti\n" if total == 2 * m else "Handshaking Theorem TIDAK
        terpenuhi\n")

    print("Adjacency list:")
    for v in G.nodes():
        print(f"  {v}: {list(G.neighbors(v))}")

    print("Adjacency matrix:")
    M = nx.to_numpy_array(G, dtype=int)
    cetak_matriks(M, list(G.nodes()))
    # jumlah angka 1 di tiap baris harus sama dengan derajat simpul itu
    print("Jumlah tiap baris :", [int(x) for x in M.sum(axis=1)])
    print("Matriks simetris  :", bool((M == M.T).all()))
    print()


analisis(nx.complete_graph(4), "Graf lengkap K4")
analisis(nx.cycle_graph(5), "Graf lingkaran C5")
analisis(nx.wheel_graph(7), "Graf roda W6 (simpul 0 = pusat)")
```

3.2 Output Program

```
===== Graf lengkap K4 =====
Derajat tiap simpul: {0: 3, 1: 3, 2: 3, 3: 3}
Jumlah derajat = 12, 2 x |E| = 2 x 6 = 12
Handshaking Theorem terbukti

Adjacency list:
  0: [1, 2, 3]
  1: [0, 2, 3]
  2: [0, 1, 3]
  3: [0, 1, 2]
Adjacency matrix:
      0  1  2  3
   0  0  1  1  1
   1  1  0  1  1
   2  1  1  0  1
   3  1  1  1  0
Jumlah tiap baris : [3, 3, 3, 3]
Matriks simetris  : True

===== Graf lingkaran C5 =====
Derajat tiap simpul: {0: 2, 1: 2, 2: 2, 3: 2, 4: 2}
Jumlah derajat = 10, 2 x |E| = 2 x 5 = 10
Handshaking Theorem terbukti

Adjacency list:
  0: [1, 4]
  1: [0, 2]
  2: [1, 3]
  3: [2, 4]
  4: [3, 0]
Adjacency matrix:
      0  1  2  3  4
   0  0  1  0  0  1
   1  1  0  1  0  0
   2  0  1  0  1  0
   3  0  0  1  0  1
   4  1  0  0  1  0
Jumlah tiap baris : [2, 2, 2, 2, 2]
Matriks simetris  : True

===== Graf roda W6 (simpul 0 = pusat) =====
Derajat tiap simpul: {0: 6, 1: 3, 2: 3, 3: 3, 4: 3, 5: 3, 6: 3}
Jumlah derajat = 24, 2 x |E| = 2 x 12 = 24
Handshaking Theorem terbukti

Adjacency list:
  0: [1, 2, 3, 4, 5, 6]
  1: [0, 2, 6]
  2: [0, 1, 3]
  3: [0, 2, 4]
  4: [0, 3, 5]
  5: [0, 4, 6]
  6: [0, 5, 1]
Adjacency matrix:
      0  1  2  3  4  5  6
   0  0  1  1  1  1  1  1
   1  1  0  1  0  0  0  1
   2  1  1  0  1  0  0  0
   3  1  0  1  0  1  0  0
   4  1  0  0  1  0  1  0
   5  1  0  0  0  1  0  1
   6  1  1  0  0  0  1  0
Jumlah tiap baris : [6, 3, 3, 3, 3, 3, 3]
Matriks simetris  : True
```

3.3 Lesson Learnt

Ketiga graf memenuhi Handshaking Theorem: jumlah derajat K4, C5, dan W6 adalah 12, 10, dan 24, masing-masing dua kali jumlah sisinya. Akibatnya jumlah simpul berderajat ganjil selalu genap, dan W6 cocok dengan itu karena punya enam simpul berderajat 3.

Adjacency matrix graf tak berarah selalu simetris, dan jumlah angka 1 di tiap baris sama dengan derajat simpulnya, jadi derajat bisa dibaca langsung dari matriks. Untuk graf jarang seperti C5, adjacency list jauh lebih ringkas karena hanya menyimpan 10 entri tetangga, sedangkan matriksnya berisi 25 sel yang kebanyakan nol.

BAB IV TASK03: TIM PROYEK

4.1 Kode Program

```
developer = ["Alyn", "Betty", "Carol", "David"]
tugas = ["UI/UX", "Backend", "Database", "Testing"]
keahlian = {
    "Alyn": ["UI/UX", "Testing"],
    "Betty": ["UI/UX", "Backend"],
    "Carol": ["UI/UX", "Database"],
    "David": ["Backend", "Testing"],
}

G = nx.Graph()
G.add_nodes_from(developer, bipartite=0)
G.add_nodes_from(tugas, bipartite=1)
for dev, daftar in keahlian.items():
    for t in daftar:
        G.add_edge(dev, t)
```

Isomorfisme diperiksa dengan membandingkan invarian graf dan `nx.is_isomorphic()`:

```
def invarian(H):
    return H.number_of_nodes(), H.number_of_edges(), sorted((d for _, d in H.degree()),
        reverse=True)


# graf khusus yang jumlah simpulnya sama-sama 8
print("\nPerbandingan dengan graf khusus (simpul, sisi, barisan derajat):")
for H, nama in [(G, "Graf tim proyek"), (nx.complete_bipartite_graph(4, 4), "K4,4"),
    (nx.cycle_graph(8), "C8")]:
    print(f"  {nama:<16} {invarian(H)}")
    if H is not G:
        print(f"  {'':<16} isomorfik dengan graf tim proyek? {nx.is_isomorphic(G, H)}")

siklus = ["Alyn", "UI/UX", "Betty", "Backend", "David", "Testing"]
print("\nSubgraf", siklus, "isomorfik dengan C6?", nx.is_isomorphic(G.subgraph(siklus),
    nx.cycle_graph(6)))

pasangan = nx.bipartite.hopcroft_karp_matching(G, top_nodes=developer)
print("\nContoh pembagian tugas (satu developer satu tugas):")
for dev in developer:
    print(f"  {dev} -> {pasangan[dev]}")
```

4.2 Output Program

```
Apakah graf bipartit? True

Adjacency list:
  Alyn: ['UI/UX', 'Testing']
  Betty: ['UI/UX', 'Backend']
  Carol: ['UI/UX', 'Database']
  David: ['Backend', 'Testing']
  UI/UX: ['Alyn', 'Betty', 'Carol']
  Backend: ['Betty', 'David']
  Database: ['Carol']
  Testing: ['Alyn', 'David']

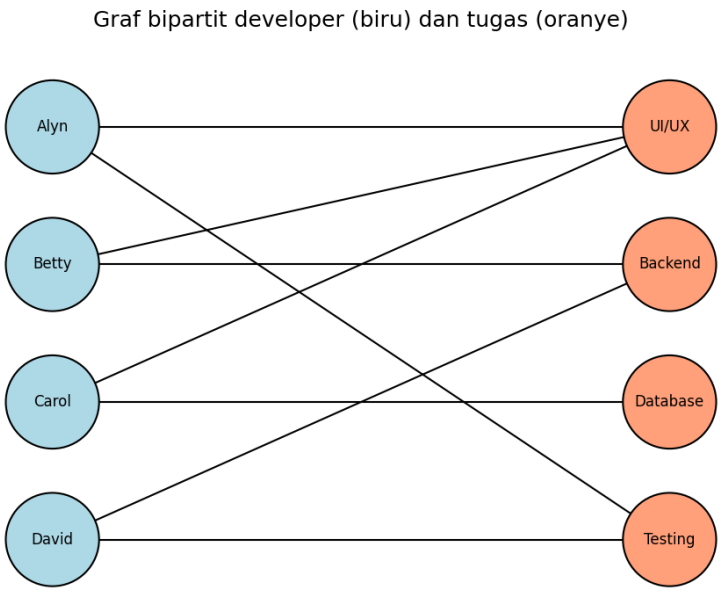
Adjacency matrix:
           Alyn  Bett  Caro  Davi  UI/U  Back  Data  Test
     Alyn     0     0     0     0     1     0     0     1
    Betty     0     0     0     0     1     1     0     0
    Carol     0     0     0     0     1     0     1     0
    David     0     0     0     0     0     1     0     1
    UI/UX     1     1     1     0     0     0     0     0
  Backend     0     1     0     1     0     0     0     0
 Database     0     0     1     0     0     0     0     0
  Testing     1     0     0     1     0     0     0     0

Derajat tiap simpul: {'Alyn': 2, 'Betty': 2, 'Carol': 2, 'David': 2, 'UI/UX': 3, 'Backend': 2,
    'Database': 1, 'Testing': 2}
Jumlah derajat = 16, 2 x |E| = 2 x 8 = 16
Handshaking Theorem terbukti: True

Perbandingan dengan graf khusus (simpul, sisi, barisan derajat):
  Graf tim proyek  (8, 8, [3, 2, 2, 2, 2, 2, 2, 1])
  K4,4             (8, 16, [4, 4, 4, 4, 4, 4, 4, 4])
                   isomorfik dengan graf tim proyek? False
  C8               (8, 8, [2, 2, 2, 2, 2, 2, 2, 2])
                   isomorfik dengan graf tim proyek? False

Subgraf ['Alyn', 'UI/UX', 'Betty', 'Backend', 'David', 'Testing'] isomorfik dengan C6? True

Contoh pembagian tugas (satu developer satu tugas):
  Alyn -> Testing
  Betty -> UI/UX
  Carol -> Database
  David -> Backend
```



Gambar 4.1: Graf bipartit developer dan tugas

4.3 Analisis dan Lesson Learnt

Graf tim proyek bipartit karena sisi hanya menghubungkan developer dengan tugas. Jumlah derajatnya 16, dua kali 8 sisi, sehingga Handshaking Theorem terbukti.

Graf ini tidak isomorfik dengan graf khusus yang sudah dipelajari. Dengan K4,4 jumlah sisinya sudah beda (8 lawan 16), dan dengan C8 jumlah sisinya sama tetapi barisan derajatnya beda: C8 semuanya 2, sedangkan graf ini punya UI/UX berderajat 3 dan Database berderajat 1. Membandingkan invarian seperti ini jauh lebih cepat untuk membuktikan dua graf tidak isomorfik daripada mencoba semua pemetaan simpul. Walaupun begitu, enam simpul Alyn, UI/UX, Betty, Backend, David, dan Testing membentuk subgraf yang isomorfik dengan C6.

Dari sisi manajer proyek, Database hanya bisa dikerjakan Carol, jadi Carol wajib memegang Database. Sisa tiga developer dan tiga tugas membentuk lintasan tertutup, sehingga hanya ada dua cara membagi tugas satu orang satu tugas, salah satunya hasil `hopcroft_karp_matching` pada output di atas.

BAB V TASK04: FULL-MESH ENAM SERVER

5.1 Kode Program

```
server = [f"S{i}" for i in range(1, 7)]
G = nx.complete_graph(server)
sisi = list(G.edges())

print(f"Jumlah server (simpul) : {G.number_of_nodes()}")
print(f"Jumlah kabel (sisi)    : {G.number_of_edges()}")
print("Daftar sisi:", ", ".join(f"e{i}={u}-{v}" for i, (u, v) in enumerate(sisi, 1)))

# incidence matrix: baris = server, kolom = sisi, isinya 1 kalau server itu ujung dari sisi
    tersebut
M = np.array([[1 if v in e else 0 for e in sisi] for v in server])
print("\nIncidence matrix:")
print("    " + "".join(f"{f'e{i}':>4}" for i in range(1, len(sisi) + 1)))
for v, baris in zip(server, M):
    print(f"{v:>3} " + "".join(f"{x:>4}" for x in baris))

M_nx = nx.incidence_matrix(G, nodelist=server, edgelist=sisi).toarray().astype(int)
print("Sama dengan nx.incidence_matrix:", bool((M == M_nx).all()))
print("Jumlah angka 1 tiap kolom (harus 2):", sorted(set(M.sum(axis=0).tolist())))
```

```
H = G.copy()
H.remove_node("S1")
print("\nSimulasi server S1 mati:")
print(f"  Sisa simpul {H.number_of_nodes()}, sisa sisi {H.number_of_edges()} (hilang
    {len(sisi) - H.number_of_edges()} kabel)")
print(f"  Masih terhubung (connected)? {nx.is_connected(H)}")
print(f"  Sisa jaringan isomorfik dengan K5? {nx.is_isomorphic(H, nx.complete_graph(5))}")
print(f"  Node connectivity K6 = {nx.node_connectivity(G)}, edge connectivity K6 =
    {nx.edge_connectivity(G)}")
```

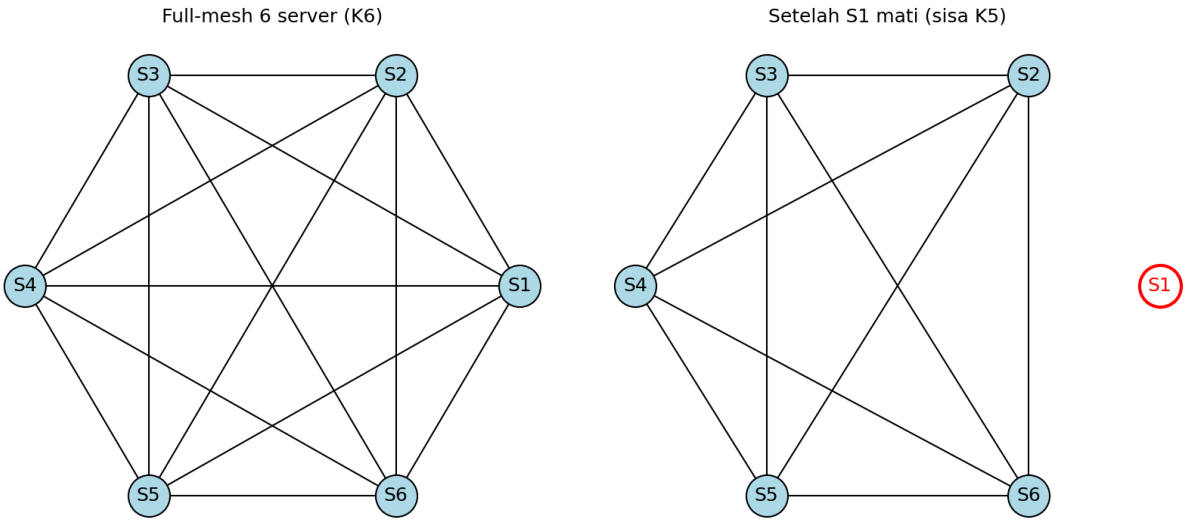
5.2 Output Program

```
Jumlah server (simpul) : 6
Jumlah kabel (sisi)    : 15
Daftar sisi: e1=S1-S2, e2=S1-S3, e3=S1-S4, e4=S1-S5, e5=S1-S6, e6=S2-S3, e7=S2-S4, e8=S2-S5,
    e9=S2-S6, e10=S3-S4, e11=S3-S5, e12=S3-S6, e13=S4-S5, e14=S4-S6, e15=S5-S6

Incidence matrix:
      e1  e2  e3  e4  e5  e6  e7  e8  e9 e10 e11 e12 e13 e14 e15
 S1    1   1   1   1   1   0   0   0   0   0   0   0   0   0   0
 S2    1   0   0   0   0   1   1   1   1   0   0   0   0   0   0
 S3    0   1   0   0   0   1   0   0   0   1   1   1   0   0   0
 S4    0   0   1   0   0   0   1   0   0   1   0   0   1   1   0
 S5    0   0   0   1   0   0   0   1   0   0   1   0   1   0   1
 S6    0   0   0   0   1   0   0   0   1   0   0   1   0   1   1
Sama dengan nx.incidence_matrix: True
Jumlah angka 1 tiap kolom (harus 2): [2]

Derajat tiap server: {'S1': 5, 'S2': 5, 'S3': 5, 'S4': 5, 'S5': 5, 'S6': 5}
Jumlah derajat = 30, 2 x |E| = 2 x 15 = 30
Handshaking Theorem terbukti: True

Simulasi server S1 mati:
  Sisa simpul 5, sisa sisi 10 (hilang 5 kabel)
  Masih terhubung (connected)? True
  Sisa jaringan isomorfik dengan K5? True
  Node connectivity K6 = 5, edge connectivity K6 = 5
```



Gambar 5.1: Full-mesh K6 dan kondisi setelah S1 mati

5.3 Analisis dan Lesson Learnt

Incidence matrix K6 berukuran 6 x 15. Setiap kolom berisi tepat dua angka 1 karena satu kabel selalu punya dua ujung, dan inilah alasan jumlah derajat pasti dua kali jumlah sisi: 6 x 5 = 30 = 2 x 15. Matriks yang disusun manual juga sama persis dengan hasil `nx.incidence_matrix()`.

Saat S1 mati, lima kabel ikut hilang, tetapi sisa jaringan tetap terhubung dan isomorfik dengan K5. Edge connectivity K6 bernilai 5, artinya satu server baru benar-benar terputus kalau kelima kabelnya putus. Kekurangannya ada di biaya: full-mesh butuh n(n-1)/2 kabel, jadi 6 server butuh 15 kabel dan jumlah itu naik kuadratik kalau server ditambah.

BAB VI TASK05: ALIRAN DATA SISTEM

6.1 Kode Program

```
aliran = [("Processor", "Memory"), ("Processor", "Storage"), ("Memory", "Processor"),
          ("Storage", "Network Interface"), ("Network Interface", "Processor")]
G = nx.DiGraph()
G.add_edges_from(aliran)

print("Adjacency list (simpul -> tujuan aliran):")
for v in G.nodes():
    print(f"  {v}: {list(G.successors(v))}")

urutan = list(G.nodes())
M = nx.to_numpy_array(G, nodelist=urutan, dtype=int)
print("\nAdjacency matrix (baris = asal, kolom = tujuan):")
print(" " * 18 + " ".join(f"{v[:4]:>5}" for v in urutan))
for v, baris in zip(urutan, M):
    print(f"{v:>17} " + " ".join(f"{int(x):>5}" for x in baris))

print("\nDerajat masuk dan keluar:")
for v in urutan:
    print(f"  {v:<18} masuk {G.in_degree(v)}, keluar {G.out_degree(v)}")
total_masuk = sum(d for _, d in G.in_degree())
total_keluar = sum(d for _, d in G.out_degree())
print(f"Jumlah derajat masuk = {total_masuk}, jumlah derajat keluar = {total_keluar}, |E| =
    {G.number_of_edges()}")
print("Handshaking Theorem versi berarah terbukti:", total_masuk == total_keluar ==
    G.number_of_edges())

# graf pembanding dengan nama simpul berbeda tapi pola aliran sama
H = nx.DiGraph([("A", "B"), ("A", "C"), ("B", "A"), ("C", "D"), ("D", "A")])
print("\nIsomorfik dengan graf A->B, A->C, B->A, C->D, D->A?", nx.is_isomorphic(G, H))
print("Pemetaan simpul:", nx.vf2pp_isomorphism(G, H))

C4 = nx.cycle_graph(4, create_using=nx.DiGraph)
print("Isomorfik dengan siklus berarah C4?", nx.is_isomorphic(G, C4))

U = G.to_undirected()
print(f"\nGraf tak berarah pembentuknya: {U.number_of_nodes()} simpul, {U.number_of_edges()}
    sisi")
print("Isomorfik dengan tadpole graph (C3 + satu simpul bandul)?", nx.is_isomorphic(U,
    nx.tadpole_graph(3, 1)))
```

6.2 Output Program

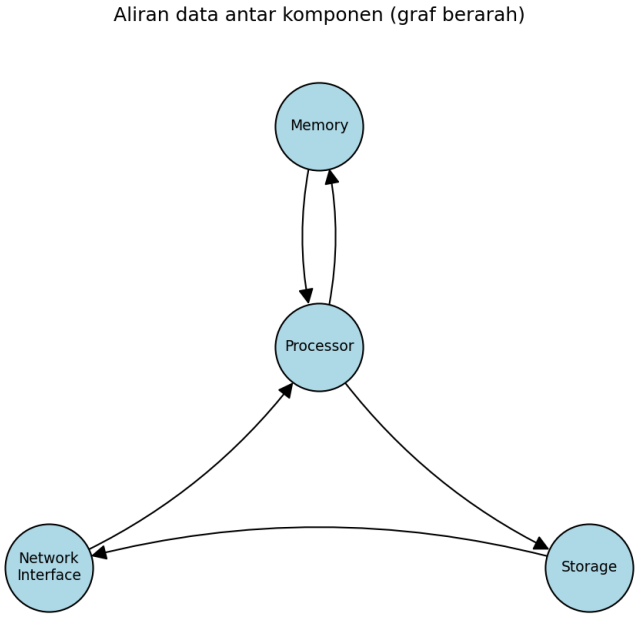
```
Adjacency list (simpul -> tujuan aliran):
  Processor: ['Memory', 'Storage']
  Memory: ['Processor']
  Storage: ['Network Interface']
  Network Interface: ['Processor']

Adjacency matrix (baris = asal, kolom = tujuan):
                   Proc  Memo  Stor  Netw
        Processor     0     1     1     0
           Memory     1     0     0     0
          Storage     0     0     0     1
Network Interface     1     0     0     0

Derajat masuk dan keluar:
  Processor          masuk 2, keluar 2
  Memory             masuk 1, keluar 1
  Storage            masuk 1, keluar 1
  Network Interface  masuk 1, keluar 1
Jumlah derajat masuk = 5, jumlah derajat keluar = 5, |E| = 5
Handshaking Theorem versi berarah terbukti: True

Isomorfik dengan graf A->B, A->C, B->A, C->D, D->A? True
Pemetaan simpul: {'Processor': 'A', 'Storage': 'C', 'Network Interface': 'D', 'Memory': 'B'}
Isomorfik dengan siklus berarah C4? False

Graf tak berarah pembentuknya: 4 simpul, 4 sisi
Isomorfik dengan tadpole graph (C3 + satu simpul bandul)? True
```



Gambar 6.1: Graf berarah aliran data antar komponen

6.3 Analisis dan Lesson Learnt

Pada graf berarah, jumlah derajat masuk sama dengan jumlah derajat keluar dan keduanya sama dengan jumlah sisi, yaitu 5, karena setiap panah punya satu pangkal dan satu ujung. Processor jadi pusat aliran dengan derajat masuk 2 dan keluar 2. Adjacency matrix-nya tidak simetris, misalnya Processor ke Storage bernilai 1 tetapi Storage ke Processor bernilai 0.

Graf ini isomorfik dengan graf A->B, A->C, B->A, C->D, D->A lewat pemetaan Processor ke A, Memory ke B, Storage ke C, dan Network Interface ke D, tetapi tidak isomorfik dengan siklus berarah C4 karena derajat masuk dan keluar Processor berbeda dengan simpul mana pun di C4. Saat diubah menjadi graf tak berarah, panah Processor->Memory dan Memory->Processor melebur jadi satu sisi sehingga 5 panah tinggal 4 sisi, dan bentuknya isomorfik dengan tadpole graph: segitiga Processor, Storage, Network Interface ditambah Memory sebagai simpul bandul.

BAB VII KESIMPULAN

1. Graf khusus bisa dibuat langsung dengan fungsi bawaan NetworkX, tetapi definisi parameternya perlu dicek, seperti `wheel_graph(n)` yang menghitung simpul pusat.

2. Handshaking Theorem terbukti pada semua graf yang dianalisis, baik tak berarah maupun berarah.

3. Adjacency list cocok untuk graf jarang, adjacency matrix memudahkan membaca derajat dan keterhubungan, dan incidence matrix memperlihatkan hubungan simpul dengan setiap sisi.

4. Perbedaan invarian seperti jumlah sisi dan barisan derajat adalah cara tercepat untuk membuktikan dua graf tidak isomorfik.